

Ride-Sharing Analytics & Driver Performance Pipeline

1. Problem Statement

Ride-sharing platforms generate large volumes of trip and driver data. However, there is no structured pipeline to analyze driver performance, cancellations, delays, and demand patterns. The task is to build a data engineering pipeline that processes raw ride data and generates meaningful insights.

2. Objectives

- Track ride and driver activity
- Analyze cancellations and delays
- Identify high-demand locations
- Evaluate driver performance
- Generate business-level KPIs

3. Architecture (Medallion Architecture)

Bronze Layer: Store raw data as-is. Silver Layer: Perform cleaning, joins, and transformations. Gold Layer: Create aggregated KPIs and analytics tables.

4. Dataset Schemas

4.1 Drivers Dataset (*drivers.csv*)

Column	Data Type	Description
driver_id	INT	Unique driver ID
name	STRING	Driver name
city	STRING	Operating city
rating	FLOAT	Driver rating

4.2 Trips Dataset (*trips.csv*)

Column	Data Type	Description
trip_id	INT	Trip ID
driver_id	INT	Driver ID
pickup_location	STRING	Pickup location
drop_location	STRING	Drop location
distance_km	FLOAT	Distance
fare_amount	FLOAT	Fare
trip_status	STRING	Trip status

4.3 Trip Logs Dataset (*trip_logs.csv*)

Column	Data Type	Description
log_id	INT	Log ID
trip_id	INT	Trip ID
start_time	TIMESTAMP	Start time
end_time	TIMESTAMP	End time
delay_minutes	FLOAT	Delay
cancellation_flag	INT	Cancellation flag

5. Tasks

1. Read all datasets using PySpark DataFrames.
2. Store raw data into Bronze layer without transformations.
3. Perform joins between drivers, trips, and trip_logs datasets.
4. Clean data by handling null values and incorrect records.
5. Create derived columns such as trip duration and completion flag.
6. Filter invalid data such as negative distance or missing timestamps.
7. Write cleaned data into Silver layer using Parquet format.
8. Generate driver performance metrics.
9. Calculate cancellation rate per driver.
10. Identify high-demand pickup locations.
11. Generate revenue-related insights.
12. Store aggregated results in Gold layer.
13. Validate data consistency across layers.
14. Apply optimization techniques in Spark.
15. Use window functions to rank drivers based on performance.